Module 1 - JavaScript

NAME : MANOSHANKARI V

SUPERSET : 6736702

1. JavaScript Basics & Setup

Objective: To understand how to connect JavaScript with an HTML page and execute basic scripts using external JavaScript files, console output, and browser alerts.

Code:

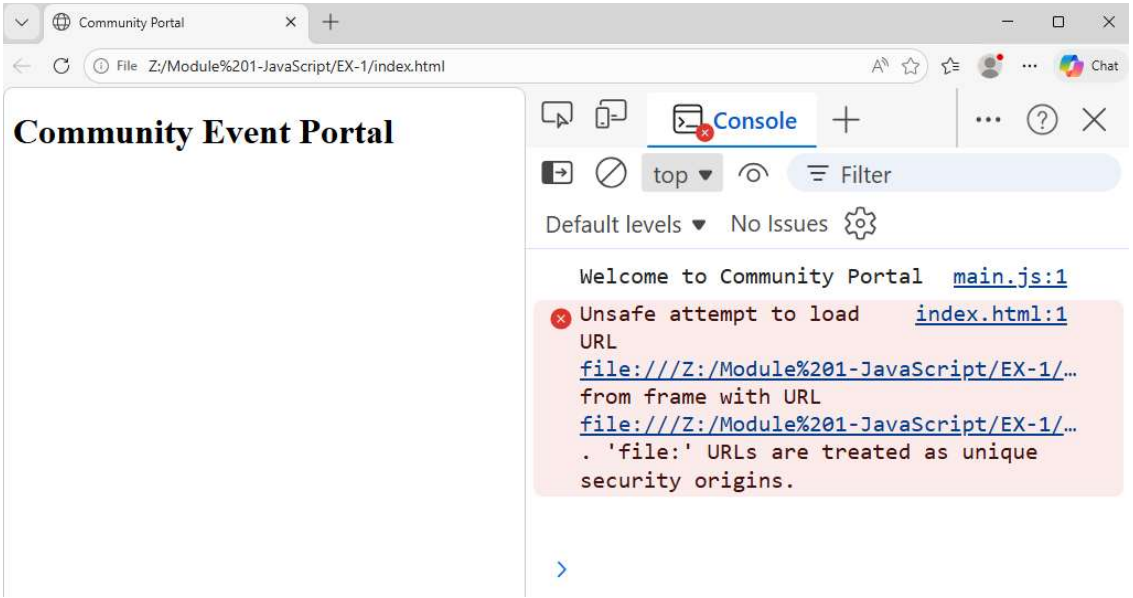
index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Community Portal</title>
</head>
<body>
<h1>Community Event Portal</h1>
<script src="main.js"></script>
</body>
</html>
```

main.js

```
console.log("Welcome to the Community Portal");
window.onload = function () {
  alert("Page fully loaded!");
};
```

Output:



2. Syntax, Data Types, and Operators

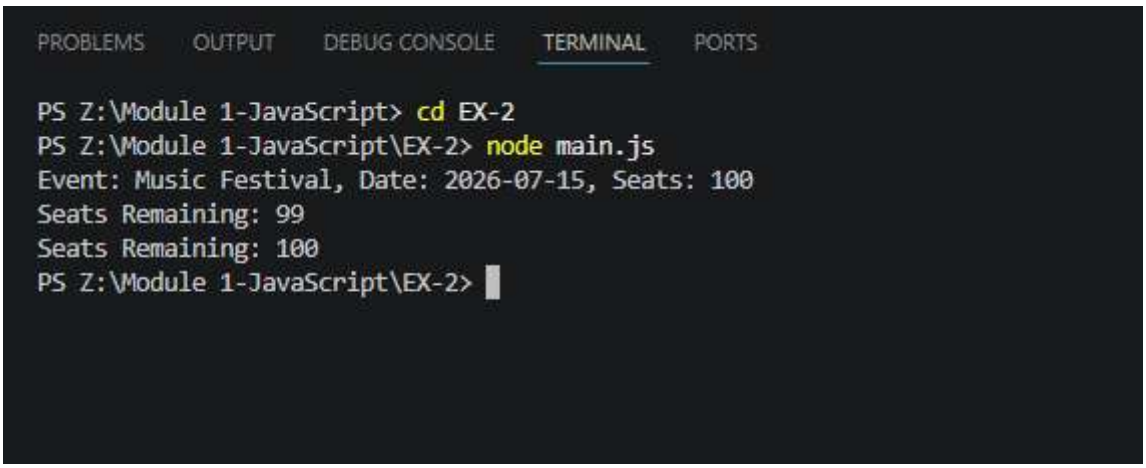
Objective: To learn how to declare variables, use different data types, apply operators, and format output using template literals.

Code:

main.js

```
const eventName="Music Festival";

const eventDate="2026-07-15";

let seats=100;

console.log(`Event: ${eventName}, Date: ${eventDate}, Seats: ${seats}`);

seats--;

console.log(`Seats Remaining: ${seats}`);

seats++;

console.log(`Seats Remaining: ${seats}`);
```

Output:



3. Conditionals, Loops, and Error Handling

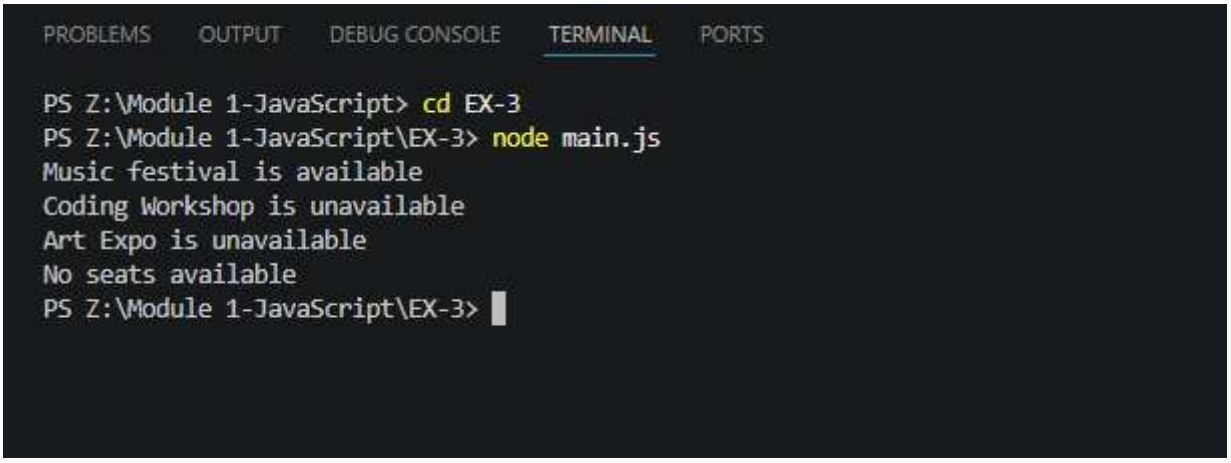
Objective: To implement decision-making, iterate through collections of data, and handle runtime errors gracefully using exception handling.

Code:

main.js

```
const events=[
   {name:"Music festival",seats:50,isPast:false},
   {name:"Coding Workshop",seats:0,isPast:false},
   {name:"Art Expo",seats:20,isPast:true}
];
events.forEach(e=>{
   if(!e.isPast && e.seats>0){
      console.log(`${e.name} is available`);
   }else{
      console.log(`${e.name} is unavailable`);
   }
});
function register(e){
   try{
      if(e.seats<=0) throw new Error("No seats available");
      e.seats--;
      console.log("Registration Successful");
   }catch(error){
      console.error(error.message);
   }
}
register(events[1]);
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS Z:\Module 1-JavaScript> cd EX-3
PS Z:\Module 1-JavaScript\EX-3> node main.js
Music festival is available
Coding Workshop is unavailable
Art Expo is unavailable
No seats available
PS Z:\Module 1-JavaScript\EX-3>

4. Functions, Scope, Closures, Higher-Order Functions

Objective: To create reusable code through functions, understand variable scope, utilize closures for data persistence, and apply higher-order functions for flexible programming.

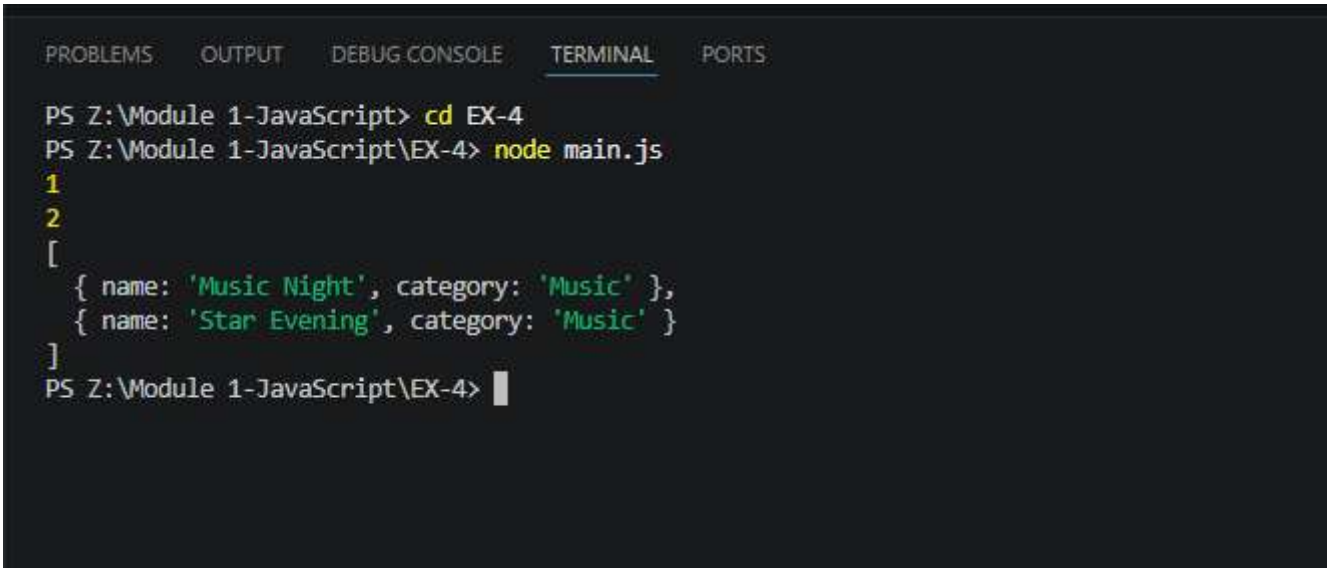
Code:

main.js

```
const events=[];
function addEvent(name,category){
   events.push({name,category});
}
function register(eventName){
   console.log(`Successfully registered  for ${eventName}`);
}
function filterEventsByCategory(category){
   return events.filter(e=> e.category===category);
}
function registrationTracker(){
   let count=0;
   return function(){
      count++;
      return count;
   };
}
const musicRegisteration=registrationTracker();
console.log(musicRegisteration());
console.log(musicRegisteration());
function searchEvents(callback){
   return callback(events);
}
addEvent("Music Night","Music");
addEvent("Star Evening","Music");
const res=searchEvents(events=> events.filter(e=>e.category==="Music"));
```

```
console.log(res);
```

Output:



5. Objects and Prototypes

Objective: To model real-world entities using objects, define constructors/classes, and extend functionality through prototypes and object methods.

Code:

main.js

```
function Event(name,date,seats){
   this.name=name;
   this.date=date;
   this.seats=seats;
}
Event.prototype.checkAvailability=function(){
   return this.seats>0;
}
const e1=new Event("Music Festival","2026-06=15",50);
console.log(e1.checkAvailability());
Object.entries(e1).forEach(([key,value])=>{
   console.log(`${key}: ${value}`)
});
```

Output:

6. Arrays and Methods

Objective: To manage collections of data efficiently using array operations such as adding, filtering, transforming, and retrieving information.

Code:

main.js

```
const Events=[];
Events.push({
   name:"Music Festival", category: "music"
});
Events.push({
   name:"Workshop on Baking", category: "workshop"
});
Events.push({
   name:"Rock Concert", category: "music"
});
const musicEvents=Events.filter(e=>e.category==="music")
console.log(musicEvents);
const cards=Events.map(e=>`Event card: ${e.name}`);
console.log(cards);
```

Output:

7. DOM Manipulation

Objective: To dynamically access, create, modify, and display HTML elements using JavaScript and update the user interface based on application state.

Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>DOM Manipulation</title>
</head>
<body>
  <div id="eventContainer"></div>
  <script src="main.js"></script>
</body>
</html>
```

main.js

```
const container=document.querySelector("#eventContainer");
const events=[
  {name: "Music Festival"},
  {name: "Coding Workshop"}
];
events.forEach(e=>{
  const card=document.createElement("div");
  card.textContent=e.name;
  container.appendChild(card);
});
function updateUI(message){
  const p=document.createElement("p");
  p.textContent=message;
  container.appendChild(p);
}
```

updateUI("User Registered");

Output:

Music Festival
Coding Workshop

User Registered

8. Event Handling

Objective: To respond to user interactions such as clicks, keyboard input, and selection changes by attaching event listeners and executing appropriate actions.

Code:

index.html

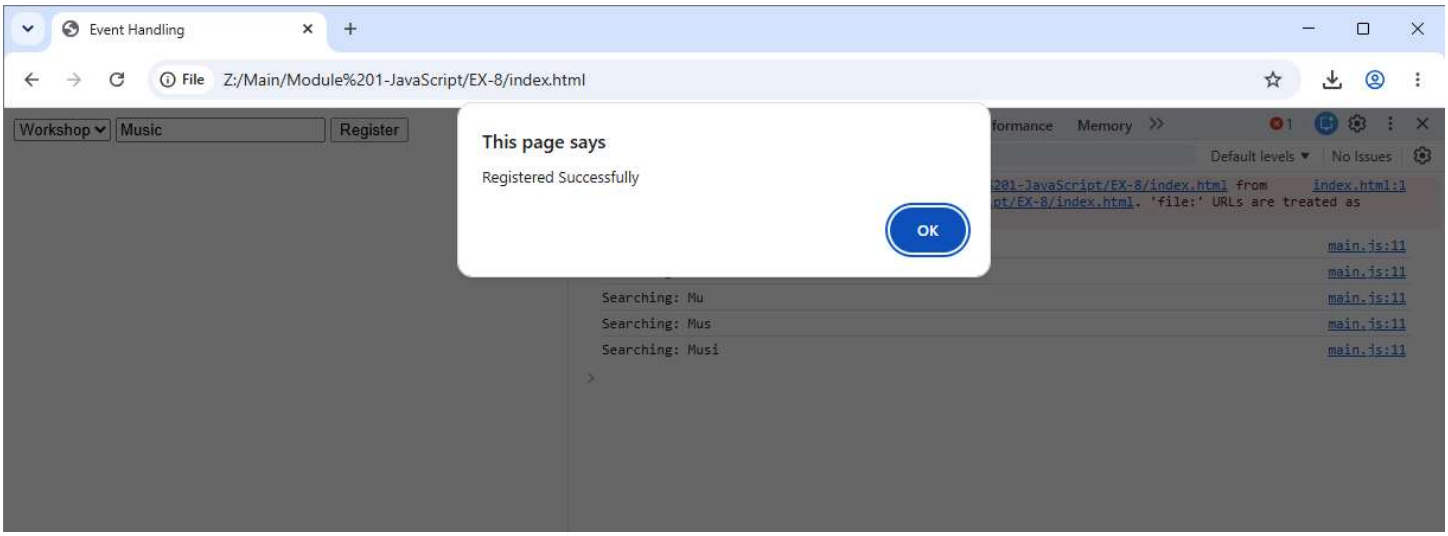
```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Event Handling</title>
</head>
<body>
  <select  id="category">
    <option >Music</option>
    <option>Workshop</option>
  </select>
  <input type="text" id="search">
  <button id="rgtrBtn">Register</button>
  <script src="main.js"></script>
</body>
</html>
```

main.js

```
document.getElementById("rgtrBtn")
.addEventListener("click",()=>{
  alert("Registered Successfully");
});
document.getElementById("category")
.onChange=function(){
  console.log("Category changed");
};
document.getElementById("search")
.addEventListener("keydown",function(event){
```

```
    console.log("Searching:",event.target.value);
});
```

Output:



9. Async JS, Promises, Async/Await

Objective: To perform asynchronous operations, retrieve data from external sources, and manage asynchronous workflows using Promises and async/await.

Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Fetch</title>
</head>
<body>
  <h1>Open Browser Console</h1>
  <div id="spinner">Loading events, please wait...</div>
  <script src="main.js"></script>
</body>
</html>
```

evensts.json

```
[
 { "name": "Music Festival" },
 { "name": "Coding Workshop" },
 { "name": "Workshop on Baking" },
 { "name": "Happening Street" }
]
```

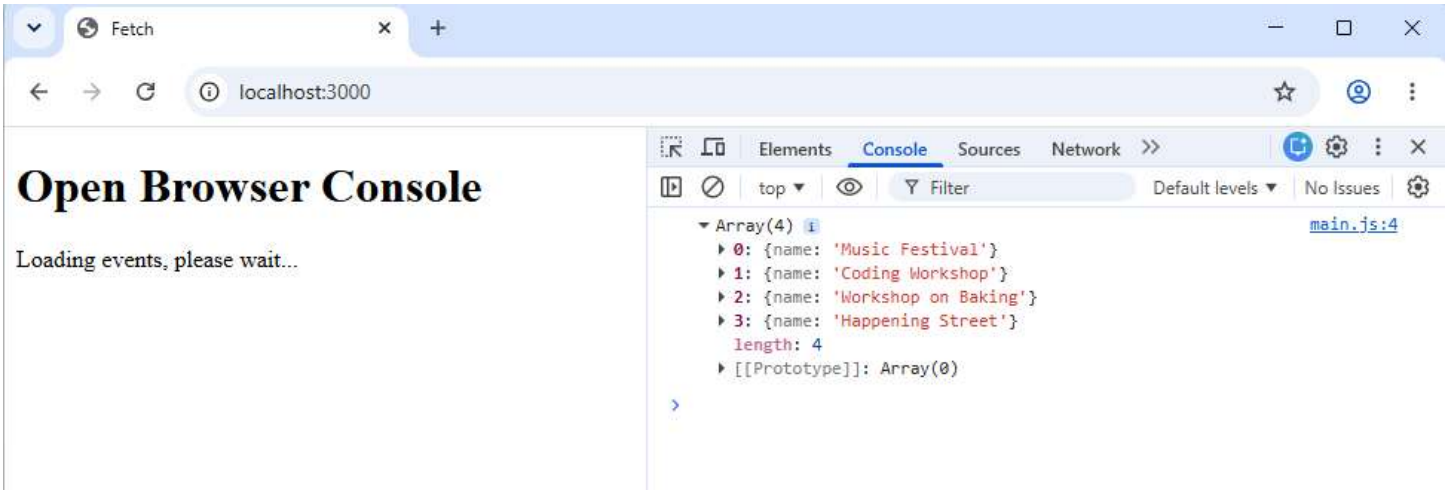
main.js

```
fetch("events.json")
.then(response=>response.json())
.then(data=>{
  console.log(data);
}).catch(error=>{
  console.error(error);    });
```

sync.js

```
async function loadEvents(){
   const spinner=document.getElementById("spinner");
   spinner.style.display="block";
   try{
      const response=await fetch("events.json");
      const data=await response.json();
      console.log(data);
   }
   catch(error){
      console.error(error);
   }
   finally{
      spinner.style.display="none";
   }
}
loadEvents();
```

Output:



10. Modern JavaScript Features

Objective: To improve code readability and maintainability using ES6+ features such as let, const, destructuring, default parameters, and the spread operator.

Code:

main.js

```
function createEvent(
   name="Unknown Event",
   category="General"
){
   return {name,category};
}
const event={
   name: "Music Festival",
   date: "2026-07-15",
   category: "Music"
};
const{name,date}=event;
console.log(name);
console.log(date);
const events=[
  { "name": "Music Festival" },
  { "name": "Coding Workshop" },
  { "name": "Workshop on Baking" },
  { "name": "Happening Street" }
];
const clone=[...events];
console.log(clone);
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS Z:\Main\Module 1-JavaScript> cd EX-10
PS Z:\Main\Module 1-JavaScript\EX-10> node main.js
Music Festival
2026-07-15
[
{ name: 'Music Festival' },
{ name: 'Coding Workshop' },
{ name: 'Workshop on Baking' },
{ name: 'Happening Street' }
]
PS Z:\Main\Module 1-JavaScript\EX-10>

11. Working with Forms

Objective: To capture, validate, and process user input from forms while preventing unwanted default browser behavior.

Code:

index.html

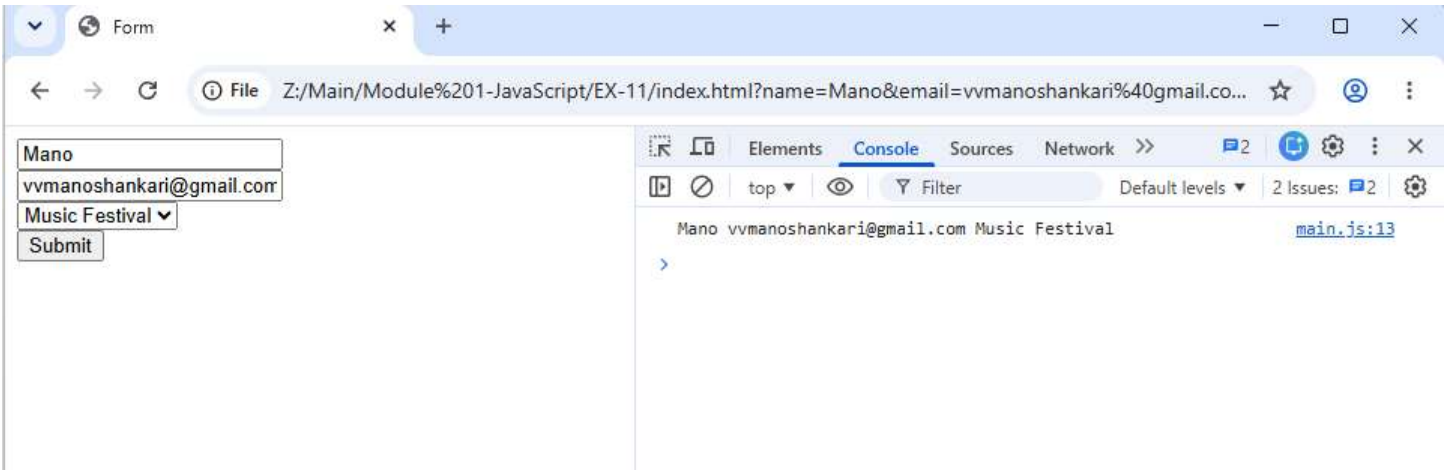
```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Form</title>
</head>
<body>
  <form id="registerForm">
    <input name="name" placeholder="Enter name"><br>
    <input name="email" placeholder="Enter email"><br>
    <select name="event">
      <option>Music Festival</option>
    </select><br>
    <button type="submit">Submit</button>
  </form>
  <div id="error"></div>
  <script src="main.js"></script>
</body>
</html>
```

main.js

```
document.getElementById("registerForm")
.addEventListener("submit", function (event) {
  event.preventDefault();
  const name=this.elements["name"].value;
  const email=this.elements["email"].value;
  const selectedEvent=this.elements["event"].value;
  const errorDiv=document.getElementById("error");
```

```
    if(!name || !email){
        errorDiv.textContent="All fields are required";
        return;
    }
    errorDiv.textContent="";
    console.log(name, email, selectedEvent);
});
```

Output:



12. AJAX & Fetch API

Objective: To communicate with backend services by sending and receiving data asynchronously using the Fetch API and handling server responses.

Code:

main.js

```
function submitRegistration(userData){
  setTimeout(async()=>{
    try{
      const res=await fetch("https://jsonplaceholder.typicode.com/posts",{
        method: "POST",
        headers:{
          "Content-Type":"application/json"
        },
        body: JSON.stringify(userData)
      });
      const data=await res.json();
      console.log("Registration Successful", data);
    }catch(error){
      console.error("Registration Failed",error);
    }
  },2000);
}
submitRegistration({
  name:"John",
  email:"john@gmail.com"
});
```

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS Z:\Main\Upskilling> cd "Module 1-JavaScript"
PS Z:\Main\Upskilling\Module 1-JavaScript> cd EX-12
PS Z:\Main\Upskilling\Module 1-JavaScript\EX-12> node main.js
Registration Successful { name: 'John', email: 'john@gmail.com', id: 101 }
PS Z:\Main\Upskilling\Module 1-JavaScript\EX-12>

13. Debugging and Testing

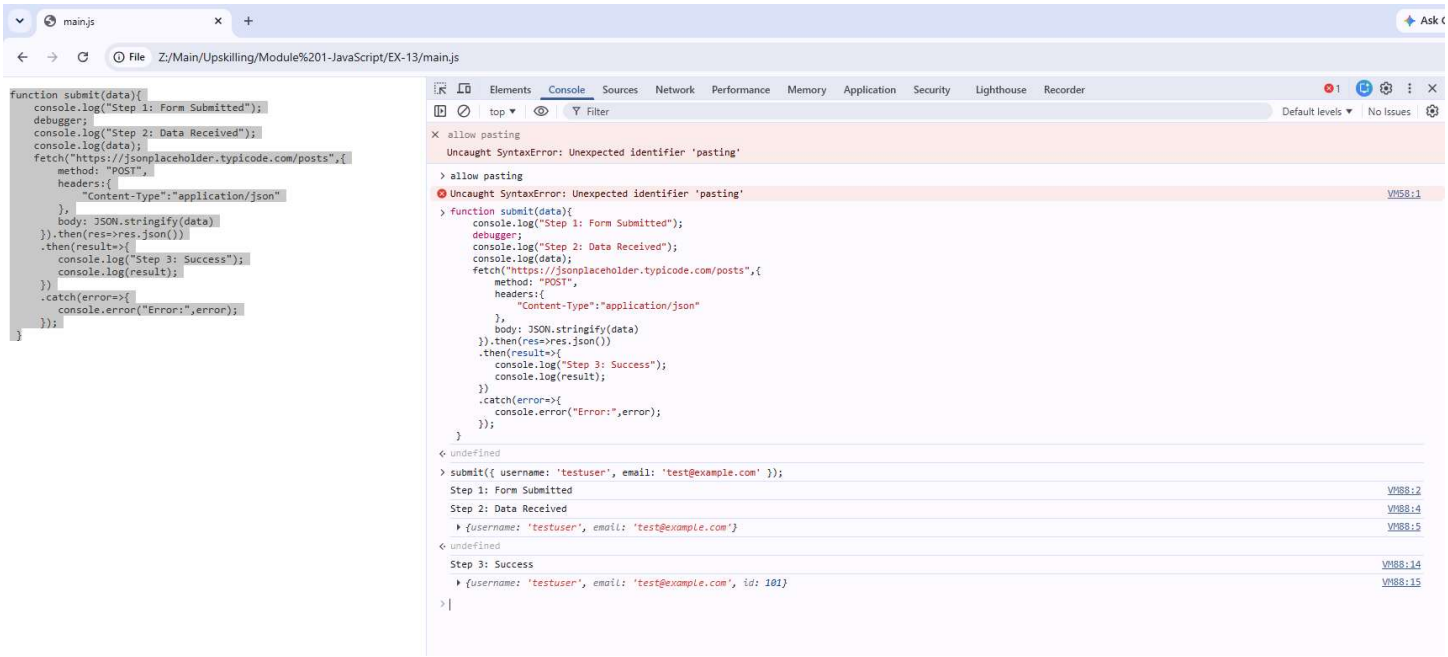
Objective: To identify, analyze, and fix errors in JavaScript applications using browser developer tools, breakpoints, logging, and network inspection.

Code:

main.js

```
function submit(data){
   console.log("Step 1: Form Submitted");
   debugger;
   console.log("Step 2: Data Received");
   console.log(data);
   fetch("https://jsonplaceholder.typicode.com/posts",{
      method: "POST",
      headers:{
         "Content-Type":"application/json"
      },
      body: JSON.stringify(data)
    }).then(res=>res.json())
    .then(result=>{
      console.log("Step 3: Success");
      console.log(result);
    })
    .catch(error=>{
      console.error("Error:",error);
    });
}
```

Output:


main.js
File Z:/Main/Upskilling/Module%201-JavaScript/EX-13/main.js
function submit(data){
console.log("Step 1: Form Submitted");
debugger;
console.log("Step 2: Data Received");
console.log(data);
fetch("https://jsonplaceholder.typicode.com/posts",{
method: "POST",
headers:{
"Content-Type":"application/json"
},
body: JSON.stringify(data)
}).then(res=>res.json())
.then(result=>{
console.log("Step 3: Success");
console.log(result);
})
.catch(error=>{
console.error("Error:",error);
});
}
Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder
top Filter Default levels No Issues
allow pasting
Uncaught SyntaxError: Unexpected identifier 'pasting'
allow pasting
Uncaught SyntaxError: Unexpected identifier 'pasting' VM58:1
function submit(data){
console.log("Step 1: Form Submitted");
debugger;
console.log("Step 2: Data Received");
console.log(data);
fetch("https://jsonplaceholder.typicode.com/posts",{
method: "POST",
headers:{
"Content-Type":"application/json"
},
body: JSON.stringify(data)
}).then(res=>res.json())
.then(result=>{
console.log("Step 3: Success");
console.log(result);
})
.catch(error=>{
console.error("Error:",error);
});
}
undefined
submit({ username: 'testuser', email: 'test@example.com' });
Step 1: Form Submitted VM88:2
Step 2: Data Received VM88:4
{username: 'testuser', email: 'test@example.com'} VM88:5
undefined
Step 3: Success VM88:14
{username: 'testuser', email: 'test@example.com', id: 101} VM88:15

14. jQuery and JS Frameworks

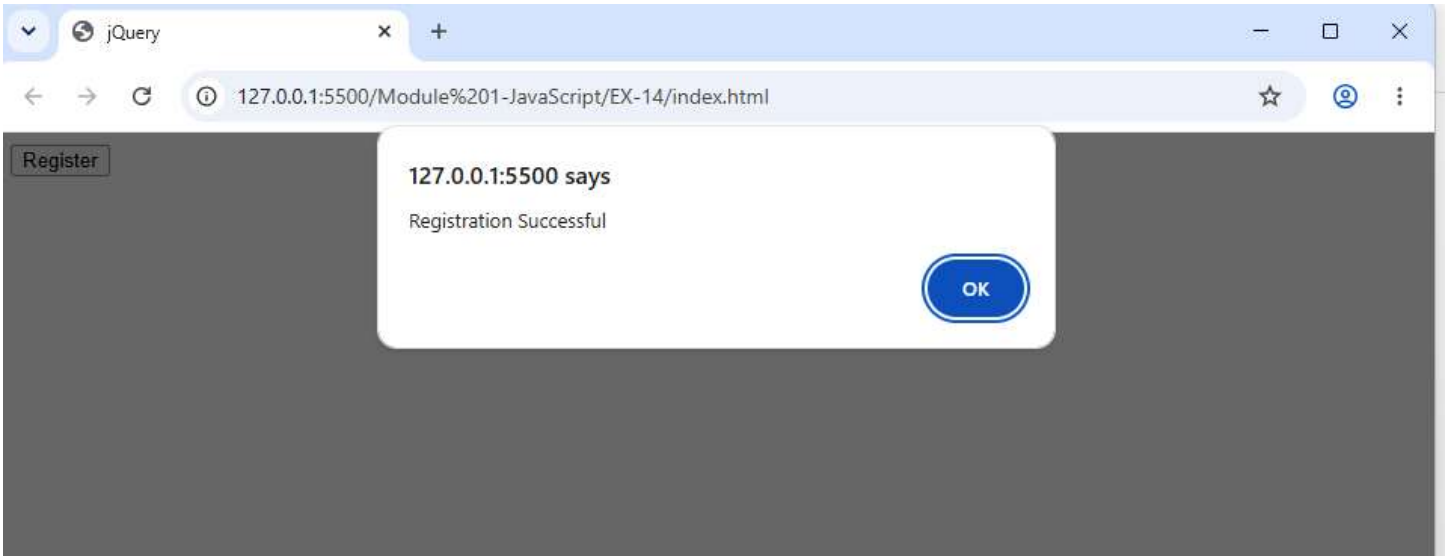
Objective: To simplify DOM manipulation and event handling using jQuery and understand the advantages of modern JavaScript frameworks such as React and Vue.

Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>jQuery</title>
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
  <button id="registerBtn">Register</button>
  <div class="eventCard">Music Festival</div>
  <script>
    $("#registerBtn").click(function () {
      alert("Registration Successful");
    });
    $(".eventCard").fadeIn();
    setTimeout(() => {
      $(".eventCard").fadeOut();
    }, 3000);
  </script>
</body>
</html>
```

Output:


jQuery
127.0.0.1:5500/Module%201-JavaScript/EX-14/index.html
Register
127.0.0.1:5500 says
Registration Successful
OK